

Azion - Technical Report

Solutions Engineer Candidate: Guilherme França

1. Executive Summary

The objective of this report is to demonstrate, based on the requirements provided, how to configure appropriately and validate the protection of a specific domain and path, while at the same time creating exclusions. The exercise is consistent with a real scenario, where specificities of each domain platform and business case drive particular rule creation.

Considering that the client has strict financial constraints, Azion's free tier and pay-per-use model makes it ideal for this scenario. Especially as this eliminates the need for separate vendors for using CDN, WAF and DDoS protection.

This full configuration was completed within a business day, which demonstrates that, even with tight deadlines, an MVP can be built and tested promptly.

As the configuration leverages the Azion's distributed platform, the traffic is routed automatically to increase performance, ensuring availability.

2. Requirements

Configure, using Azion's platform, an application to be used as a reverse proxy, pulling data from <https://httpbingo.org>.

- Enable WAF exclusively on /login* to mitigate common attacks
 - Including POST / GET
- Bypass WAF on static files
- Implement aggressive caching for the same static files.

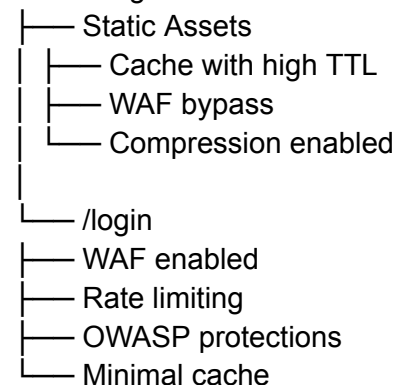
3. Architecture Overview

When a user tries to access this website, it will be resolved to an Azion edge server. In this instance: <https://fklsxzla7sy.map.azionedge.net>

User

↓

Azion Edge



4. Security and Cost justifications

Azion Edge Application	[gfranca-test.map.azionedge.net] Application Accelerator enabled This module reduces the connection failure changes and improves resilience. It was required to implement granular caching rules for static assets, improving origin offload. Security Justification: Centralizes traffic through Azion edge servers, Improves Traffic control, Cost Justification: Reduces origin traffic through caching Reduces backend processing
Connector	[gfranca-test.map.azionedge.net] - Pointing to the origin server httpbingo.org Security Justification: The initial configuration was made to enforce HTTPS redirection.
Rules Engine	[StaticFiles-Caching] Used the following regex to cover an increased amount: <pre>\.(css js mjs map json xml txt tff woff woff2 svg svgz jpg jpeg gif bmp png webp avif ico mp4 webm ogg mp3 wav pdf zip gz br)(\?.*)?\$</pre> <i>Gzip compression enabled at the edge for static assets.</i>
Firewall	[Enabling WAF for login Path] Security Justification: Protects the high-risk endpoints (login page) against major attack threats. [ByPass Firewall] Using a WAF rule without security controls in place, with the match: <pre>\.(css js mjs map json xml txt tff woff woff2 svg svgz jpg jpeg gif bmp png webp avif ico mp4 webm ogg mp3 wav pdf zip gz br)(\?.*)?\$</pre>

	[Rate Limiting] When Uri starts with /login Rate Limit of 5 req/s -> This prevents brute force credential attacks while accommodating legitimate users. Maxburstsize: 10 -> Enough to accommodate concurrent background resources when a browser fetches files (css, JS, images).
WAF Rules	[WAF-LoginProtection] Security Justification: Protects the high-risk endpoint (login) - SQL Injection [Enabled/High] - Login pages require high protection against these common attacks. - RFI [Enabled/Medium] - Less likely to happen on Login pages, but catches server side injection attempts. - Directory Transversal [Enabled/Medium] - Prevents path manipulation in login parameters. - XSS [Enabled/High] - Attackers can attempt to execute code to steal credentials. Should be high on login pages. - File Upload [Disabled] - Login endpoints don't usually accept file uploads. - Unwanted access [Enabled/Medium] - Medium should be sufficient to catch scanning patterns towards /login. - Evading Tricks [Enabled/High] - Certainly needs to be enabled as high/highest. Attempting to bypass WAF rules. - Identified Attacks [Enabled/High] - Catches known attacks like sqlmap. Cost Justification: WAF scoped only to /login which reduces inspection on all requests, across all other traffic.
Cache Settings	Static assets cached for 30 days. Security Justification: /Login is set to 0s TTL at the edge to prevent sensitive information being cached. Cost Justification: 30-day TTL for static assets significantly reduces origin requests and bandwidth costs.